Fast Parallel Construction of Bounding Volume Hierarchies Project

MAËL RIOS, Institut Polytechnique de Paris, France

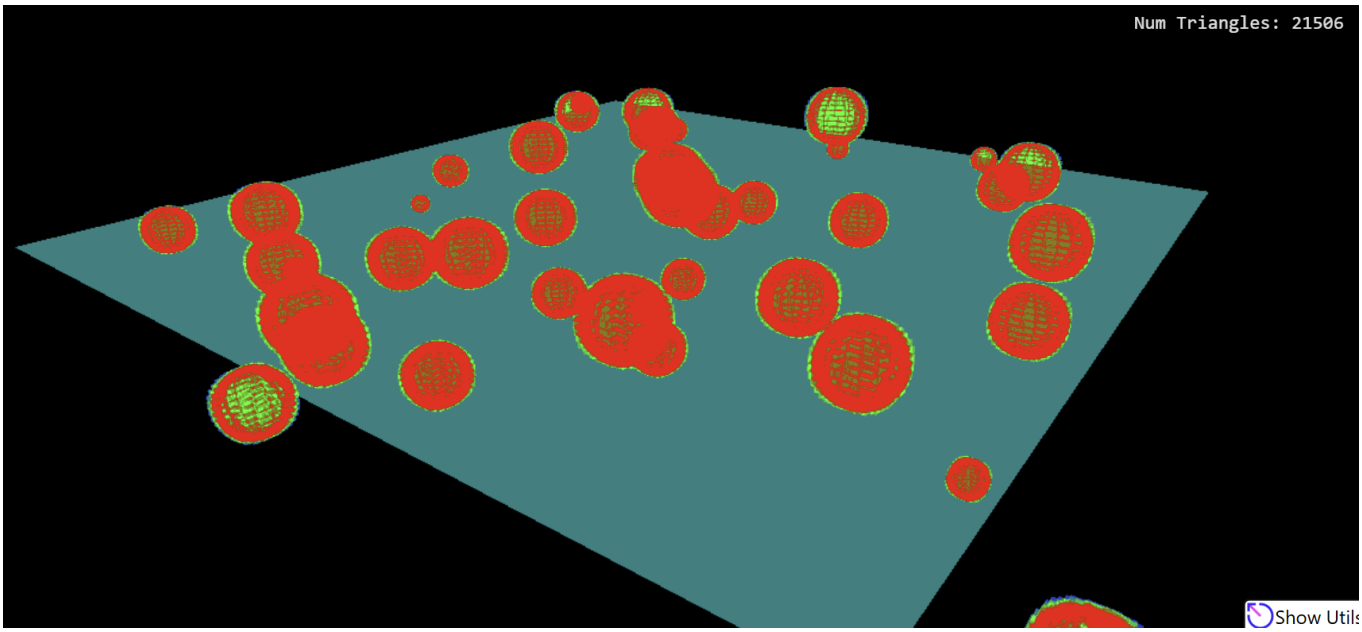

Fig. 1. Rendering of the constructed BVH's leaves, converging to the same BVH leaves as my SAH sequential binned cpu BVH

ACM Reference Format:
Maël Rios. 2026. Fast Parallel Construction of Bounding Volume Hierarchies Project. 1, 1 (March 2026), 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Ray tracing is very powerful when it comes to real-time rendering. Microfacet BRDF can be inferred just like any rasterization pipeline, the shadows from multiple light sources are very easy to compute, and it allows for some impressive Physically Based Rendering such as reflections and refraction with multiple ray bounces.

This however comes at a high cost. In a naive implementation, each ray needs to test against every single triangle in the scene in order to decide what part of a mesh has been hit, and you need to reproduce this for every shadow traced and every light bounce. Therefore, this method is not scalable to bigger scenes with multiple hundred of thousands of triangles.

Thankfully, there are structures that can encapsulate the triangles in the scene in order to hash the space the scenes occupies. A Bounding Volume Hierarchy (BVH) is such a structure, and this way, if the partitioning of the scene is done correctly, we can accelerate the time taken by a ray to query the "boxes" of the structure up to the triangle they encapsulates that intersects the ray.

In bigger scenes with meshes containing thousands of hundreds of triangles, the BVH can accelerate the time taken to render a frame by hundreds of times in high end GPUs, which is why this structure is often used in the industry for state of the art real time rendering pipelines.

2 Two ways of approaching BVH

In the assignments, I was able to implement a SAH (Surface Area Heuristics) binned BVH computed at the initialization of the scene geometry on each mesh I added. This produced a very high quality BVH, per individual mesh, that immediately allowed me to have, in a low-end computer GPU, 60 FPS on scenes with more than 200 000 triangles. However, this BVH is computed only once at the start of the scene, and the only movements "allowed" in the scene to not alter the BVH are the transforms I applied to the per mesh world matrices that allowed me to move the BVHs and the triangles in the scene and not recompute them.

There are cases where we would like to have different BVH at different time steps of our application. We can imagine for example the vertices of a mesh moving in a way that a world transform matrix is not sufficient to handle the precomputed BVH, or when we want to deal with a lot of rigidbody collisions in the most parallel

Author's Contact Information: Maël Rios, Institut Polytechnique de Paris, France.

© 2026 ACM.
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in , https://doi.org/10.1145/nnnnnnn.nnnnnnn.

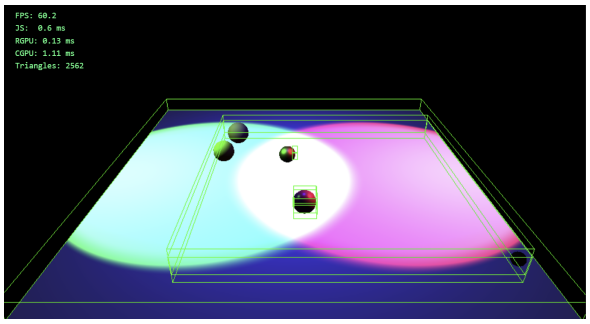

Fig. 2. Example of BVH nodes rendered in wireframe at depth 3, with multiple spheres in different nodes.

way possible. In these cases, a reconstruction of the BVH has to be done in between frames, which is very expensive (my SAH binned BVH for example is implemented in CPU, and porting it to GPU every frame is not a possibility if we want to have an interactive application).

This is where the paper I decided to implement comes in. In Karras 2012[2], they present a method to maximize the parallelization throughput of GPUs to construct (low quality, but still functional) space hashing structures such as Octrees or BVH. In milliseconds, a BVH is produced and since it is all done with compute shaders, we can directly link it to the rendering shader.

Then comes Karras 2013[3] that doubles down on the output of this method by proposing an also parallel way to convert the low-quality BVH into one that can rival with high-quality producing methods.

This report contain summaries and WebGPU implementation details for the main important algorithms in those papers. I will first present the pipeline that outputs a low quality BVH from the vertex buffer, and then the optimizations proposed to augment it's quality described in [3].

3 Maximizing Parallelism in the Construction of BVHs

The goal of the project was to implement a wgsl compute shader pipeline that can take the buffer of vertices in world space and output a scene wide BVH. That way, linking a single final BVH buffer to the renderer makes our application ready for its traversal.

When reading the shaders, and the javascript code to link resources together, you will see the usage of workgroup shared memory and thread specific instructions often to maximize the parallel power of wgsl. For example, I will often use multiple threads for processing data on a buffer and reduce it with each time less and less threads until only one thread ends up writing the result we were looking for: the local workgroup ID for each thread is often used as a sign of "should I be doing work or not" in order to avoid overwriting the shared memory on race conditions.

I will describe the compute pipeline as I have implemented it, in order, and will incorporate details from the paper [2] whenever they are important to understand the methods used.

3.1 scene min/max AABB

The first couple of compute shader I implemented is a min/max AABB sweep over the scene with reduction over local minima/maxima to find the scene global bounds. This is done with two simple shader passes that will allow us to store the min and max vertices positions in the scene, which will be useful to normalize each vertex world position axis between [0, 1] in preparation for the morton code sorting.

The second shader reduces the written min/max position of the vertices in order to output a single global scene bounding box, which we pass along the next pipeline.

3.2 Morton Codes

Once the global bounding box is computed, I implemented a shader that dispatches a thread per triangle in the scene and computes it's morton code per axis (x, y, z). In the paper, they recommend using 60 bit morton codes for better precision on the bounds of each vertex. However, to facilitate the implementation of the radix sort and respect WebGPU constraints, I used 30 bit precision morton codes (10 bits per axis) which is more than enough to validate conceptually the method and get good results.

These morton codes will become the keys in the sorting next pass, and the triangle indices become the values of the keys we store in order to identify the correct triangles in the scene once the morton codes are sorted.

There is no real technicality in this pass, computing a morton code from a position normalized by the bounds of the scene is easy, so the gain here is that we are able to dispatch one thread per triangle to maximize parallel efficiency.

3.3 Radix Sort

Radix sort is a very powerful parallel sort algorithm that could, by itself, be the subject of a paper implementation on its own. In addition, nailing the radix sort was the first thing I did in the process of completing the project, as being certain that the algorithm works by itself is good grounds on which we can build the rest of the pipeline.

I am particularly proud of my implementation of a clean self-contained radix sort demo you can find in this link. The benchmarks on a RTX4070 GPU that show how useful this is to speed up sorting by up to 50x compared to builtin javascript sorting cpu-side can are presented in Table 1.

I implemented a 4-way radix sort (2 bits per pass, hence 4 buckets) following [1]. For 30-bit Morton codes, the algorithm repeats the same procedure over 15 passes. At each pass, it extracts the current 2-bit group from every code, assigns each element to one of four buckets corresponding to 00, 01, 10, or 11, and then performs a prefix-sum step to compute the correct offsets of each bucket. These

Table 1. Benchmark comparison between built-in JavaScript CPU sorting and the WebGPU radix sort implementation on an RTX 4070. Times are reported in milliseconds. The speedup factor is computed as CPU Avg/Radix Avg.

# Elements	CPU Min	CPU Max	CPU Avg	Radix Min	Radix Max	Radix Avg	Speedup
1 000	0.00	0.20	0.10	2.90	4.90	3.16	0.03×
10 000	1.10	1.60	1.24	2.70	4.90	3.15	0.39×
100 000	14.70	16.70	15.01	4.10	7.10	5.30	2.83×
500 000	84.30	89.40	85.97	5.20	8.40	6.49	13.25×
1 000 000	177.90	185.00	180.29	8.30	15.40	11.40	15.81×
5 000 000	983.90	1016.00	995.17	14.20	27.00	20.64	48.22×
10 000 000	2077.10	2118.90	2090.62	26.60	170.10	37.45	55.82×
20 000 000	4327.90	4634.60	4374.70	76.60	85.10	79.29	55.17×

offsets are then used to reorder the codes in the output buffer before proceeding to the next pair of bits. After the 15 passes, we end up with the codes completely sorted.

To do so, I use three compute shaders. The first computes the local bucket prefix-sum per thread. The second one performs a hierarchical blelloch scan (composed of a reduce operation then an addition operation) to get the correct offsets which are then used by the third shader. That last shader adds the local bucket rank and global block offset together, which ends up giving the new reorder position of the code. This is done with a double-buffering ping pong in between 2 buffers, which means that after 15 passes (needed to scan the 30 bit morton codes) we end up with the results on the second buffer. The values buffer (containing the indices, and will output sorted indices) is also given to the shader so that they are sorted accordingly to how the keys are reordered.

Radix sort is an essential pass in the construction of the Binary Radix (also called Patricia) tree[2] as the morton codes that generate it need to be ordered for the following algorithm to work.

3.4 The Patricia Tree

The low-quality BVH constructed from [2] will be a binary tree, where each leaf represents exactly one triangle in the scene. The tree is constructed in such a way that it resembles a trie structure on our morton codes: the internal nodes correspond to prefixes of a 30 bit code, and its children all have that prefix at the beginning of their code. We construct a tree with N - 1 internal nodes and N leaves, where N is the number of triangles in the scene.

At construction time, each internal node can figure out its range and children using what is called a δ function. In an ordered binary radix tree, an internal node covers a lexicographic key range $[i, j]$ whose common prefix length is $\delta(i, j)$, where $\delta(i, j)$ denotes the longest common prefix of k_i and k_j. The range is partitioned at a split index $g \in [i, j-1]$ satisfying $\delta(g, g+1) = \delta(i, j)$, giving us child ranges $[i, g]$ and $[g+1, j]$.[2]

This can be done in parallel assigning one thread per internal node I_i: we exploit the node layout, where an internal node index coincides with either the first or last key of its range. For each $i \in [0, n-2]$, the algorithm first determines whether the node range extends to the left or right (this is the direction d in the shader) by comparing the common-prefix lengths of its neighbors. This gives a direction $d \in \{-1, +1\}$ and a lower bound $d_{\min}$ that separates the node from its sibling. The other end of the range is then found by refining a directional step with binary search to obtain the maximal range endpoint j. Once the full range $[i, j]$ is known, the split position γ is found by a second binary search using the node prefix length $\delta(i, j)$. The left and right children are finally assigned from γ and $\gamma + 1$, referencing either a leaf or an internal node depending on whether the corresponding subrange contains one single key or multiple keys. This algorithm is described in algorithm 1.

What is more, the paper gracefully takes care of the special collision cases where we get the same morton code for two leaves: they "augment" each key with a bit representation of their index $k'_i = k_i \oplus i$.

With a single pass, dispatching one thread per internal node, we now have a patricia tree representation of the BVH, but we still have to compute each node's AABB by propagating the spaces taken by each triangle on the leaves in a bottom-up manner until we find the root.

```
for all internal nodes with index i ∈ [0, n − 2] in parallel do
    // Determine direction of the range (+1 or −1)
    d ← sign(δ(i, i + 1) − δ(i, i − 1))
    // Compute upper bound for the length of the range
    d_min ← δ(i, i − d)
    l_max ← 2
    while δ(i, i + l_max d) > d_min do
        l_max ← 2 l_max
    // Find the other end using binary search
    l ← 0
    for t ∈ {l_max/2, l_max/4, ..., 1} do
        if δ(i, i + (l + t)d) > d_min then
            l ← l + t
    j ← i + ld
    // Find the split position using binary search
    d_node ← δ(i, j)
    s ← 0
    for t ∈ {⌈l/2⌉, ⌈l/4⌉, ..., 1} do
        if δ(i, i + (s + t)d) > d_node then
            s ← s + t
    g ← i + sd + min(d, 0)
    // Output child pointers
    if min(i, j) = g then
        left ← L_g
    else
        left ← I_g
    if max(i, j) = g + 1 then
        right ← L_{g+1}
    else
        right ← I_{g+1}
    I_i ← (left, right)
```

Fig. 3. Algorithm pseudocode taken from [2] directly, explaining in detail the parallel construction of the patricia tree.

3.5 AABB propagation

One of the last steps once the Patricia tree is computed, is to get the AABB bounds of each internal node, necessary when constructing a BVH.

In the paper, they describe an algorithm and method that they will reuse in karras 2013 [3], which is the following. We keep an array buffer (size of N-1) of atomic counters, that we have to reset each pass, and assign a thread per leaf node. Then, each thread will climb up the tree from that leaf until they find an unvisited internal node. The first thread to reach it bails out and writes to the atomic counter, informing the second arriving thread that it has written the aggregation information (AABB min/max based on its children) and that this second thread should continue climbing the tree. Following this logic, only one thread ends up reaching the root node, which also means all AABB have been constructed for all internal nodes.

The paper implements this method using CUDA, which means that they have access to macros and features (like acquire) that guarantee the child internal nodes had their AABB written before aggregating on the parent. I tried to port this 1-1 to WebGPU and it was a failure... the tree would break half the time, and no proper aggregation was constructed from this, because of race conditions.

I spent a long time trying to figure this out before finding a LBVH WebGPU implementation on github (by AddisonPrairie) that apparently had the same issues as me because they had a very weird fix that magically makes the bottom-up pass work. They actually store the AABB aggregations as atomic integers, and if any of the axes is 0 when the parent tries to read, that means that the thread at the child node has not finished writing yet. In that case, the thread at the parent continues in an infinite loop until it reads a change in the buffer. This is quite dangerous, and not a very elegant way to treat the problem, but I do not think there is any alternative if we want to code this in wgsl...

I implemented that pattern inside my pipeline and it ended up working. We now have a proper low quality BVH in just few milliseconds, but it is not yet in the format we expect for traversal.

3.6 DFS Flattening

One last aspect that puzzled me was the traversal of the BVH as seen in class. Ideally, we would not want to use a stack traversal inside a compute shader. The idea is to have the BVH flattened in DFS format such that we can do the traversal with a pointer to the root, and instructions on how much do we increment the pointer in the case of a hit or a miss (often called a miss link). This is way cleaner and has a specific end condition: the pointer is equal to the number of total nodes.

Therefore, I wrote a final compute pass which flattens the BVH tree in DFS format, which is parallel since the patricia tree pass allowed us to get the range of each internal node, i.e. the number of nodes in it's subtree.

The reason it works in parallel is because we assign each node its final DFS position directly, rather than generating the order by exploring the tree sequentially. For any internal node or leaf, its DFS index is computed by walking upward through its parent chain to the root. At each ancestor, if the current node lies in the left child, only the parent itself appears before it in DFS order, so the index is incremented by 1, which is what we do in the traversal when visiting the left children, +1 to the current BVH node index. If it lies in the right child, then the DFS traversal must first visit the parent and the entire left subtree, so the index is incremented by $1 + |T_{\text{left}}|$, where $|T_{\text{left}}|$ is the number of nodes in the left subtree. This is the miss link index we use in the traversal. Since each node can independently recover how many nodes precede it in DFS order from only its ancestors and the precomputed subtree sizes, every thread can write its node directly to the correct slot of the flat array, and we get our final DFS ordered BVH ready to be used in the raytracing shader!

4 High-Quality BVH

I will now explain in detail the intrinsics of the algorithm karras 2013[3] proposes to upgrade the quality of the output low quality BVH from the previous detailed pipeline.

4.1 Some Implementation Details

I did not implement this part of the paper, not because I did not try to, but because similarly to how I needed to deviate from the paper in the AABB reconstruction shader, I realized implementing this algorithm in WebGPU was near impossible if we wanted to stay true to what is described. I did not have enough time to rewrite this part once I found the niche solution for the AABB reconstruction shader.

This paper implementation details heavily rely on well known CUDA features such as the atomic write and acquire guarantee I talked about earlier, subgroups BALLOT and SHUFFLE used to recruit near threads to help on the computation of treelets (per-warp optimization), and other details I realized were not available in WebGPU, or very poorly documented / experimental.

I probably spent more than 30-35 hours trying to make it work, most of my work can be found in the shader treeletOptimization.wgsl but I had to give that up to finalize the project and at least have a working low-quality BVH we can visualize and use.

4.2 The Algorithm

The key takeaway of this paper is that we are able to upgrade the quality of the BVH, with input a low quality LBVH. Thanks to the use of the Surface Area Heuristic (SAH), we restructure parts of the tree around internal nodes in parallel, which changes the ordering of their children. By doing this, we push next to each other nodes that are closer in space by altering the internal topology of the tree, which ends up minimizing the SAH cost (this is the main heuristic that gave me a high quality BVH with my binned sequential algorithm). Furthermore, a post processing pass collapses nodes together when in need, so we do not end up having one leaf per triangle, but some leaves can now posess multiple triangles (i.e. faster to query triangles in that node than to query the otherwise following children nodes). This is done as a separate pass after the treelet optimization passes which needs to keep the one triangle per leaf structure in order to work.

```
// Calculate surface area for each subset
for s̃ = 1 to 2^n − 1 do
    a[s̃] ← AREA(UNIONOFAABBS(L, s̃))
// Initialize costs of individual leaves
for i = 0 to n − 1 do
    c_opt[2^i] ← C(L_i)
// Optimize every subset of leaves
for k = 2 to n do
    for all s̃ ∈ [1, 2^n − 1] with k set bits do
        // Try each way of partitioning the leaves
        (c_s̃, p̃_s̃) ← (1, 0)
        for all p̃ ∈ {partitionings of s̃} do
            c ← c_opt[p̃] + c_opt[s̃ ⊕ p̃]                ▷ S \ P
            if c < c_s̃ then
                (c_s̃, p̃_s̃) ← (c, p̃)
        // Calculate final SAH cost (Equation 2)
        t ← TOTALNUMTRIANGLES(L, s̃)
        c_opt[s̃] ← min(C_i · a[s̃] + c_s̃, C_t · a[s̃] · t)
        p̃_opt[s̃] ← p̃_s̃
```

Fig. 4. Finding the optimal tree using dynamic programming. L is an ordered sequence of the n treelet leaves, and $\tilde{s}$ and $\tilde{p}$ are bitmasks representing subsets of these leaves. The algorithm processes subsets in increasing order of size. The optimal SAH cost of each subset is stored in c_{opt}, and the corresponding partitioning is stored in $\tilde{p}_{\text{opt}}$. The full tree cost is given by $c_{\text{opt}}[2^n - 1]$. Taken directly from [3]

To do this, they define a treelet as a subtree, with exactly K leaves (which can be internal nodes, they are just defined as the leaves of the treelet). To ease comprehension, and stay true to my implementation, we use the recommended number of 7 leaves per treelet. Where the naïve way would be to compute the score of every single internal restructuring of the treelet, a more elegant way to proceed is to solve sub-problems (2 leaves in the set, then 3, etc...) in order to solve the treelet optimization problem containing the 7 leaves. A subset can be seen as a 7-bit code, where bit $i = 1$ means the leaf i belongs to the current sub-problem we solve, $i = 0$ otherwise. We start by solving subsets with 2 leaves, and end up reaching the final subset to optimize which would be $s = 1111111 = 127$, corresponding to all leaves included in the final optimization problem. This is a Dynamic Programming algorithm, which can be found in detail in algorithm 2.

$$C(n) = \min \begin{cases} C_i\, A(n) + C(n_l) + C(n_r), & n \in Internal, \\ C_t\, A(n)\, N(n), & n \in Leaves. \end{cases} \tag{1}$$

For a treelet with $K = 7$ leaves, the compute shader only evaluates the $2^K - 1 = 127$ leaf subsets and, for each subset, stores the best achievable SAH cost and corresponding split. This avoids repeatedly recomputing equivalent subproblems that would arise when testing every possible tree shape independently. Once we find the best possible split and restructuring of the treelet, we write it out and jump on to the next treelet. The final SAH cost of every subset is described in equation 1, which also gives us the frontier to determine whether

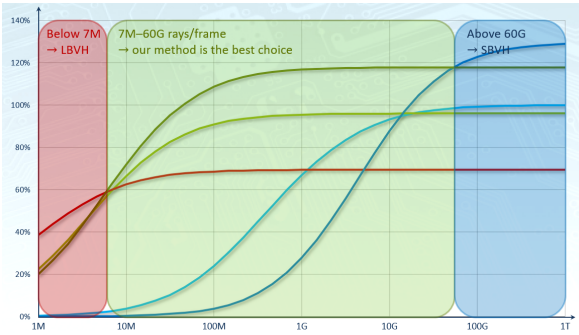

Fig. 5. Effective performance versus number of rays per frame, illustrating the trade-off between faster construction and higher-quality BVHs. Taken from [3].

we collapse this node or not in the post-processing pass.

In order to do this in parallel, the paper proposes the same bottom-up with atomic counters algorithm as they used in the AABB reconstruction of the patricia tree. We start with one thread per leaf, with subgroup dispatching, such that when a thread encounters a treelet to optimize (based on a γ heuristic, such that if $numTrianglesInSubtree >= \gamma$, it becomes a treelet), the thread recruits (switching from per thread traversal to per-warp treelet optimization) the other ones in it's subgroup through BALLOT and SHUFFLE in order to ensure a strict bottom-up traversal of the tree and ensure no race conditions corrupt children-parent pointers. Furthermore, this guaranties the treelets in the descendants have been optimized, and maximizes SIMD utilisation to make the DP practical in GPU (thanks to the CUDA features that I was not able to reproduce in WebGPU).

The paper recommends doing this optimization pass three times before node collapsing, and doubling the value of γ at each pass seemed to produce better results. This makes sense as we would start by restructuring treelets that are down in the tree in the first pass and we would end up restructuring the topology of internal nodes near the top of the tree at the end (similar to converging to the optimal SAH partitioning of the tree topology).

The final recommendation done by the paper is to split triangles occupying a large surface area into multiple nodes. This extra processing step done once at the beginning, before the treelets optimization, gives better results on the final optimization. You can already see this effect in my demo when activating / deactivating the ground, producing a somewhat of a better quality LBVH: the rendering takes less time to complete without the ground there.

5 Conclusion

5.1 Paper conclusions

That concludes the presentation of the two papers, in their most brief details. karras 2012 [2] gives us a massively parallel way to compute fast low quality bounding volume hierarchies, in such a way that it is possible to compute them each frame for the entire scene. karras 2013[3] gives us an extra algorithm that sacrifices some of the time taken to compute the BVH for better performances in the raytrace pipeline by upgrading the quality of the BVH. The graphic in figure 3 ends up giving us the sweet spot of when we should be using this algorithm, sorted by effective-performance/number of rays ratio. The paper covers a wide range, which makes it (or at least made it in 2013) a very useful approach on parallel BVH construction in the field.

5.2 My personal notes

I initially chose this paper because I had a lot of fun writing the binned SAH BVH algorithm in javascript, and seeing the performance boost it gave me in the Raytracer and Path Tracer was extremely satisfactory. Furthermore, I believe Radix Sort is an extremely important parallel sorting algorithm I had to learn to implement, and reading this paper was worth it for that alone, since I was able to correctly implement a minimal radix sort compute pipeline in WebGPU.

However, I hit a lot of very frustrating walls during the implementation of the project, and spent many many hours trying to port the optimization paper into WebGPU when it was clearly thought as a CUDA implementation with many macros and features I was not familiar with and that I had never used in my life. I still do not know if not being able to plug in the treelet optimization pass was more of a WebGPU limitation than it was my lack of knowledge with CUDA compute shaders, and the time I spent trying to fix that was time lost that I could have spent making a better demo perhaps. Compared to using my precomputed Binned SAH BVH, this low quality BVH ends up giving much worse performances, which is frustrating because it was way harder to implement. Now that I found a workaround for the problem I had on the AABB reconstruction, I believe that with some more time I can end up wiring up the optimization pipeline to finish the project in my personal time.

It was still not in vain, as I learned a lot from reading the paper and writing it out in wgsl. Figuring things out in parallel, optimizing the use of the threads and proper dispatching was very fun and taught me many things I would have otherwised never touched with a more "rendering focused" paper. This was a challenge for myself, and even though the final render speed is disappointing, it gave me a lot of useful tools I will be able to reuse in the future.

We might not have in the end a very high quality BVH every frame, but we are able to compute one for 50 000 triangles under 5 ms on consumer laptop GPU!

References

[1] Linh Ha, Jens Krüger, and Cláudio T. Silva. 2009. Fast Four-Way Parallel Radix Sorting on GPUs. *Computer Graphics Forum* 28, 8 (2009), 2368–2378. doi:10.1111/j.1467-8659.2009.01542.x

[2] Tero Karras. 2012. Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees. In *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*. 33–37. doi:10.2312/EGGH/HPG12/033-037

[3] Tero Karras and Timo Aila. 2013. Fast parallel construction of high-quality bounding volume hierarchies. In *High Performance Graphics*. https://api.semanticscholar.org/CorpusID:14653555